

HPC Project Report

Leonardo Antonio Riola, Davide Donati, Giacomo Girotto

1 Assignment 1

1.1 Introduction and objectives

The goal of this assignment is to implement a simulator for the propagation of a damped wave, governed by the following partial differential equation:

$$\frac{\partial^2 u}{\partial t^2} + \gamma \frac{\partial u}{\partial t} = c^2 \nabla^2 u \quad (1)$$

where $u(x, y, t)$ is the wave height on a two-dimensional plane over time, γ is the damping coefficient, and c is the propagation speed. The simulator computes u at every point (x, y) as t advances, with the final goal of producing an MP4 video of the simulation.

1.2 Implementation

The continuous model is discretised on a regular grid: space is divided into points spaced by Δx and Δy , and time into steps of Δt . Each grid point is identified by indices (i, j) and each instant by an index t , so that $u(x, y, t)$ becomes a matrix of values $u_{i,j}^t$. The derivatives in the equation are approximated with **central differences method** [13], so that the next wave state $u_{i,j}^{t+1}$ is:

$$u_{i,j}^{t+1} = \frac{1}{\gamma \Delta t + 1} \left[c^2 \Delta t^2 \left(\frac{u_{i+1,j}^t - 2u_{i,j}^t + u_{i-1,j}^t}{\Delta x^2} + \frac{u_{i,j+1}^t - 2u_{i,j}^t + u_{i,j-1}^t}{\Delta y^2} \right) + \gamma u_{i,j}^t \Delta t - u_{i,j}^{t-1} + 2u_{i,j}^t \right] \quad (2)$$

The temporal evolution is handled with three $M \times M$ matrices, **u_past**, **u** and **u_fut**, dynamically allocated with **malloc** and representing the wave state at times $t - 1$, t and $t + 1$ respectively. All elements are initialised to zero except for a single impulse at the centre of the grid $(M/2, M/2)$, used to start the perturbation. The impulse amplitude, propagation speed and damping coefficient used throughout this work are:

Impulse	Propagation speed c	Damping coefficient γ
-49	0.33	0.187

The core of this assignment is the parallelization of the update loop with **OpenMP**. A **#pragma omp parallel** region is opened once before the time loop, and the two nested spatial loops are distributed across threads with **#pragma omp for schedule(static) collapse(2): schedule(static)** splits the iterations into equal-sized chunks assigned before execution [8], while **collapse(2)** fuses the i and j loops into a single $M \times M$ iteration space [5], giving the scheduler far more, finer-grained work units to balance across cores. Each thread thus computes $u_{i,j}^{t+1}$ for a disjoint set of coordinates, independently and concurrently with the others.

Once **u_fut** has been fully computed for the current timestep, it must be written to a PGM file; this is done inside a **#pragma omp single** block, so that only one thread performs the write while the rest wait at the implicit barrier. To avoid the cost of deep-copying entire matrices between timesteps, the temporal advance is implemented as a three-way pointer rotation (**u_past** takes the address of **u**, **u** takes that of **u_fut**, and **u_fut** is recycled for the next iteration), performed by the same thread that just wrote the file. This keeps the pointer swap race-free and avoids any extra synchronization beyond the barrier already required for the write.

1.2.1 Identified inefficiencies and corrective transformations

All profiling and testing in this section was performed using the `Edu_skylake` partition (2 nodes, 32 physical and logical cores) [1] with 32 threads and a workload size of $M = 1000$. The aim was to identify the main bottlenecks and improve the source code.

We profiled the initial implementation with **Intel VTune Profiler**. *Hotspots* analysis identifies which functions in an application consume the largest share of execution time, using low-overhead sampling to build a ranked list of hotspot functions as a starting point for optimization [2]. *Threading* analysis instead evaluates how efficiently the application uses the available processor cores, exposing inefficiencies in threading-runtime usage and contention at synchronization points that leave threads idle and prevent full processor utilization, summarized through its *Effective CPU Utilization* metric [3]. Combining the two analyses lets us separate genuine computation time from time lost to idle spinning at synchronization points.

Hotspot analysis showed that `gomp_team_barrier_wait_end` and `fwrite` accounted for 79.4% and 9.7% of total CPU time respectively, against only 2.2% for the matrix computation itself: the bottleneck was therefore not the stencil, but excessive synchronization and I/O overhead. Three successive transformations addressed this, each verified either via VTune or direct code inspection.

Improvement 1: buffered frame output. Frames were originally written pixel by pixel with `fwrite()` inside a single-threaded `#pragma omp single` block. We replaced this with an in-memory buffer, filled in parallel via a dedicated `#pragma omp for` and flushed to disk with one `fwrite()` call, cutting I/O calls from one per pixel to one per frame. This did introduce a new barrier, however, since the buffer-fill loop is a separate worksharing construct from the stencil loop.

Improvement 2: loop fusion. Since the stencil and the buffer-fill iterate over the exact same (i, j) range, we merged them into a single `#pragma omp for`, writing each scaled pixel immediately after computing `u_fut[i][j]`. This removes the extra barrier introduced above, with no loss of parallel work.

Improvement 3: parallel initialization and scheduling policy. The matrix initialization, originally performed sequentially by a single thread, was parallelized with `#pragma omp parallel for`. By Amdahl’s law, this serial fraction would otherwise cap the achievable speedup regardless of how well the main loop scales. On a multi-socket compute node, each CPU socket has its own directly-attached memory bank: cores access memory local to their own socket faster than memory attached to another socket, a design known as NUMA (Non-Uniform Memory Access) [7]. Under the default *first-touch* policy, a physical page is assigned to the socket of whichever thread writes to it first, we therefore gave the initialization loop the same schedule as the stencil loop, so that each thread’s first-touch region matches the region it repeatedly accesses afterwards. Profiling the complete pipeline (including I/O) showed, however, that `schedule(guided)` outperforms `schedule(static)` in practice ¹: under `guided` scheduling, chunks are handed out to threads on request rather than fixed in advance, with each chunk’s size proportional to the number of remaining iterations and shrinking toward the specified minimum (1 as default) as the loop progresses [5]. This dynamic redistribution better compensates for the load imbalance introduced by the single-threaded frame-writing step, so `guided` was retained for the final implementation. As shown later in the testing section, isolating the computation from I/O reverses this result, confirming that NUMA locality does matter once the synchronization overhead of I/O is removed.

After these three improvements, VTune Hotspot analysis showed the CPU time spent in `gomp_team_barrier_wait_end` dropping to 57.5%, while time spent in the matrix computation itself rose to 15.7%; I/O (`fwrite`) time fell below 0.3% ². The Threading analysis, however, revealed that the program was still using only 23% of the available CPUs, for an average effective utilization of 7.362 out of 32 threads: a clear sign that, at $M = 1000$, the workload was simply too small for the

¹See `output_1920835.txt` (static) and `output_1920842.txt` (guided)

²See `output_1920920.txt`

number of threads requested. This motivated the next round of tests, which vary the workload size and other parameters to see how resource utilization changes.

1.3 Testing and Results

The remaining tests were run on the `Edu_sapphire` partition (2 nodes, 48 physical and 96 logical cores each); each node is dual-socket, with 24 physical cores per socket [1].

1.3.1 Sweep of workload size

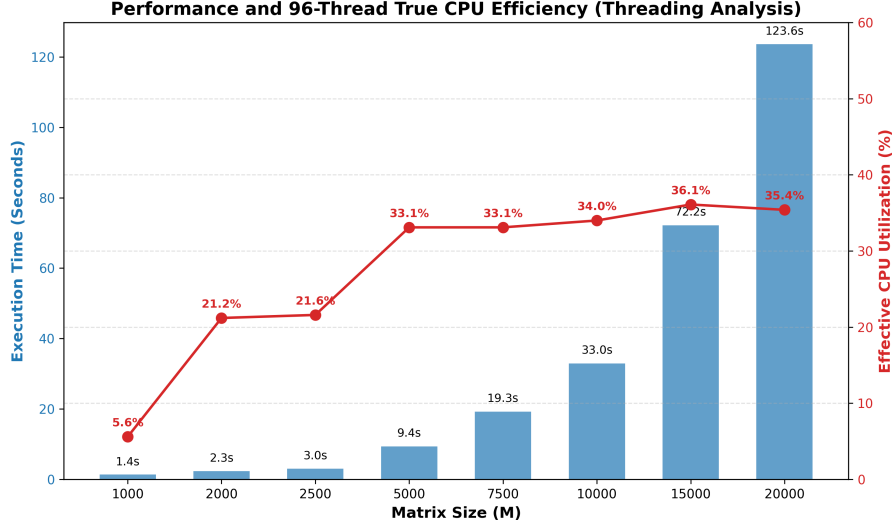


Figure 1: Execution time and true CPU efficiency scaling with varying grid sizes (M), using 32 threads.

In order to investigate the architectural limits of the system, a workload sweep test was conducted using the maximum logical capacity of the compute node (96 threads), varying the grid size M from 1,000 to 20,000. Figure 1 illustrates the execution time (extracted without the profiling overhead) vs the True Effective CPU Utilization, extracted from the VTune threading analysis. The utilization curve reveals three distinct operational phases. For small grids ($M = 1000$ to 2500), the system suffers from severe *thread starvation*, where the computational workload is too small to justify the distribution across 96 threads and two NUMA sockets, resulting in an initial efficiency plateau around 21%, dominated by synchronization overhead, barrier wait times and I/O operations. As M increases ($M = 5000$ to 15000), the computational density grows quadratically ($O(M^2)$) and in accordance with Gustafson’s Law, the useful computational work begins to outweigh the fixed parallelization overhead, driving the CPU utilization upward. However, the efficiency strictly plateaus at approximately 36% for massive grids ($M \geq 15000$). This hard limit is the superposition of three architectural constraints: the strict sequential bottleneck of the disk I/O operations (Amdahl’s Law), the saturation of the memory bandwidth (memory wall), and the effects of Hyper-Threading. Regarding the latter, since the node possesses only 48 physical cores, through Hyper-Threading each core exposes two logical processors to the OS, which share the same execution units, caches and memory bandwidth [4]. So, attempting to utilize 96 threads for a memory-bound stencil operation forces logical threads to heavily compete for the same physical cache resources, preventing the utilization from approaching 100%. Regarding the absolute execution times, the data perfectly validates the theoretical $O(M^2)$ spatial complexity of the 2D grid simulation. For instance, doubling the grid dimension from $M = 10000$ to $M = 20000$ effectively quadruples the computational domain and the execution time scales from 33.0 seconds to 123.6 seconds, achieving a 3.74x increase that is remarkably close to the theoretical 4x factor. Conversely, in the smaller grid range ($M \leq 2500$), the constant overhead associated with managing 96 concurrent threads, along with the fixed costs of implicit OpenMP barriers, heavily dominates the actual execution time, confirming the thread starvation phenomenon observed in the efficiency curve.

1.3.2 Sweep of number of threads at $M = 15000$

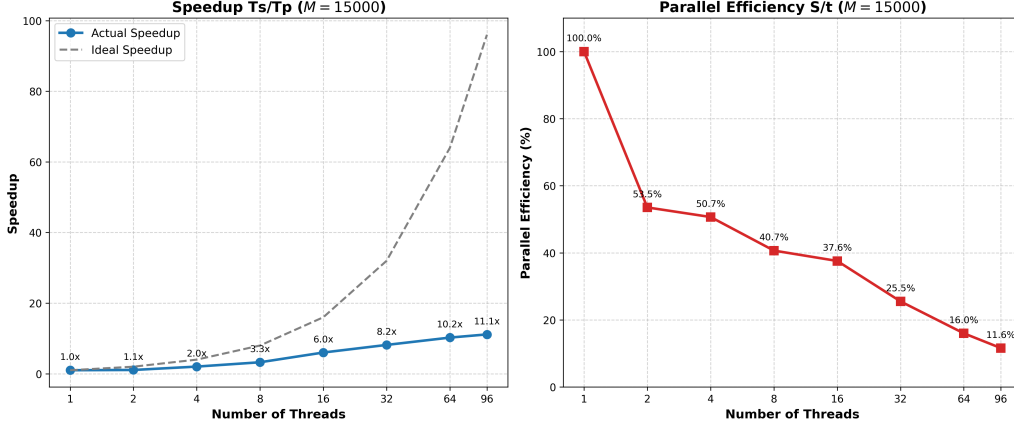


Figure 2: Speedup and parallel efficiency with varying number of threads.

We then tested the program with a fixed workload size of $M = 15000$ varying the number of threads and collected the **Speedup** and **Parallel Efficiency** (Speedup/Number of threads) parameters. Figure 2 shows two distinct regimes. For $P \leq 32$, all threads run on independent physical cores; the efficiency loss is explained by the OpenMP barriers cost, which grows with team size. For $t = 64$ and $t = 96$, thread count exceeds the number of physical cores (48), so Hyper-Threading is used from that point onward.

We repeated the same test with the I/O operations disabled, so that the program only computes the matrix updates.³

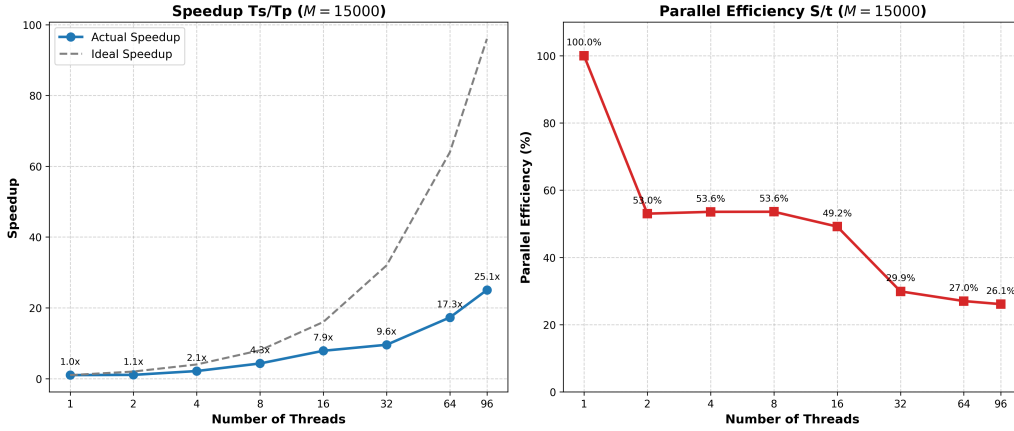


Figure 3: Speedup and parallel efficiency with varying number of threads, I/O disabled.

As Figure 3 shows, both metrics improve significantly: at $t = 96$ speedup more than doubles, from $11.1\times$ to $25.1\times$, and parallel efficiency grows from 11.6% to 26.1% . This confirms that a substantial share of the overhead seen in the full version came from the I/O operations and their associated synchronization barriers, rather than from the stencil computation itself.

Finally, we replaced `schedule(guided)` with `schedule(static)` in both the initialization and the stencil loop, to restore the NUMA alignment mentioned earlier.⁴ Once thread count exceeds 24, work is necessarily split across both sockets, and accesses to memory owned by the other socket incur the added latency of the inter-socket interconnect.

³See output_1925400.txt, output_1925398.txt, output_1925396.txt, output_1925394.txt, output_1925399.txt, output_1925397.txt, output_1925395.txt, output_1925393.txt in folder "results_noio"

⁴See output_1928239.txt, output_1928222.txt, output_1928220.txt, output_1927019.txt, output_1927027.txt, output_1927036.txt in folder "results_noio-static"

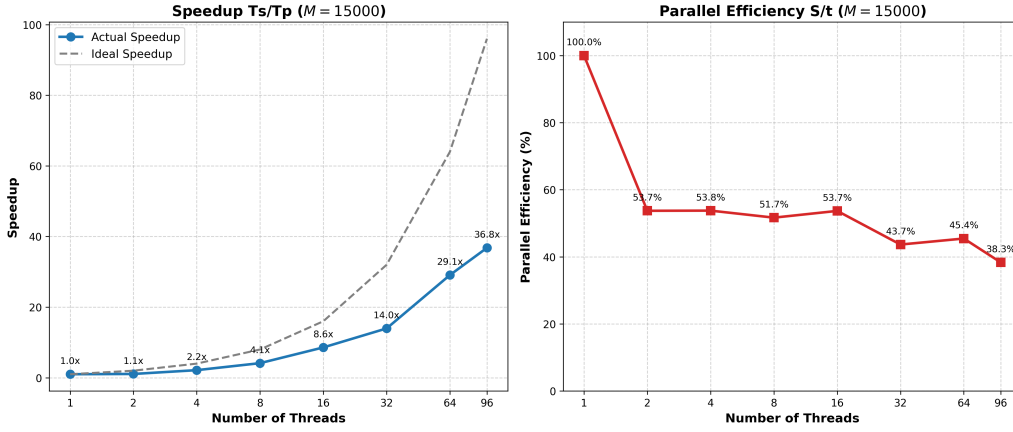


Figure 4: Speedup and parallel efficiency with varying number of threads, I/O disabled, `schedule(static)`.

As shown in Figure 4, the static schedule substantially mitigates this effect: the efficiency drop between $t = 16$ and $t = 32$ is roughly halved (from -19.3 to -10.0 percentage points relative to guided), and efficiency remains markedly higher at $t = 32$ (43.7% vs. 29.9%), $t = 64$ (45.4% vs. 27.0%, a 68% relative gain) and $t = 96$ (38.3% vs. 26.1%), with speedup at $t = 96$ rising from $25.1\times$ to $36.8\times$. This is consistent with the static schedule preserving the same thread-to-row mapping established during initialization, which keeps most memory accesses local to each thread’s own socket even as the workload spans both NUMA domains.

2 Assignment 2

2.1 Introduction

Following the initial wave simulator developed with OpenMP, this second assignment extends the application to execute multiple concurrent simulations on an HPC system using a hybrid MPI and OpenMP programming model. The goal is to study multi-wave interaction and interference dynamics under different physical conditions. While OpenMP continues to handle data parallelism by accelerating the 2D stencil computations across local CPU cores, MPI is introduced to manage task parallelism. Specifically, the system executes three distinct simulation scenarios simultaneously within a single execution run:

- **Simulation 1 (Rank 0):** Reproduces the baseline single-wave scenario initiated by a single central impulse at coordinates $(M/2, M/2)$ at $t = 0$.
- **Simulation 2 (Rank 1):** Models wave superposition and interference phenomena by initiating two simultaneous impulses at different spatial locations at $t = 0$.
- **Simulation 3 (Rank 2):** Introduces time-delayed interference dynamics by generating a second impulse at a spatial location after a predetermined time-shift ($t = t_{start}$).

By decoupling these independent workloads across separate MPI processes, the hybrid architecture maximizes computational throughput without requiring inter-process communication during the time-stepping loop.

2.2 Implementation

Building upon the code developed in Assignment 1, the implementation extends the simulator to handle multiple concurrent runs. The core numerical scheme for the time-stepping propagation and the mapping function converting field values into grayscale images (`write_matrix`) remain identical to the first version, ensuring consistency in wave physics and visualization.

The execution starts by initializing the MPI environment with **MPI_Init**, retrieving the total number of processes with **MPI_Comm_size**, and identifying the rank ID of each process with **MPI_Comm_rank**. To run the three required simulations simultaneously, the program verifies that at least 3 processes are allocated. Task parallelism is managed through conditional branching based on the process rank, where each rank specifies its scenario parameters before calling a dedicated **run_simulation** function that encapsulates the core time-stepping loop.

While data parallelism within each process is handled by OpenMP, the execution incorporates rank-specific logic. Thread synchronization constructs, such as the **pragma omp single** directive, are carefully utilized within the time loop to safely manage tasks that must be executed by a single thread—such as array pointer swapping, injecting the delayed impulse, and saving frames—preventing race conditions without halting the entire team of threads unnecessarily.

2.3 Test and Optimization

A fundamental optimization in this hybrid architecture concerns memory management. Each MPI process independently allocates memory for its local grid arrays (**u_past**, **u**, and **u_fut**) utilizing a custom **matrix_alloc** function. Although there is no inter-process communication between the MPI ranks in this specific decoupled setup, contiguous memory allocation remains crucial for the local OpenMP threads. It maximizes spatial locality and hardware prefetching efficiency, ensuring that the CPU cache is utilized optimally during the 2D stencil computations. This approach significantly helps to mitigate the heavy contention on the main RAM caused by the three concurrent processes.

A second crucial aspect is minimizing thread synchronization overhead. While data parallelism is handled by OpenMP, tasks that must be executed by a single thread, such as array pointer swapping and saving frames, require synchronization constructs like the **pragma omp single** directive. To optimize this, the injection of the time-delayed impulse for Simulation 3 in the rank 2 was carefully structured. Instead of placing the conditional check inside a synchronized block, the evaluation (**if (rank == 2 && t == n_start)**) is placed strictly *outside* the **pragma omp single** directive. This architectural choice ensures that the threads only encounter the synchronization barrier at the exact time step when the impulse must be applied. During all other time steps, and for all other simulation ranks, the threads completely bypass the barrier, preventing race conditions without halting the entire team of threads unnecessarily and drastically reducing idle times.

Following these code optimizations, the execution times of the three ranks were profiled, executed with 8 OpenMP threads per rank. As expected from a well-balanced workload, the execution times show no significant variation among the three processes, demonstrating excellent load balancing despite the ranks having different initial conditions and logic branches. However, when comparing these results to the baseline times from Assignment 1, a severe performance degradation is observed as the matrix size (M) increases.

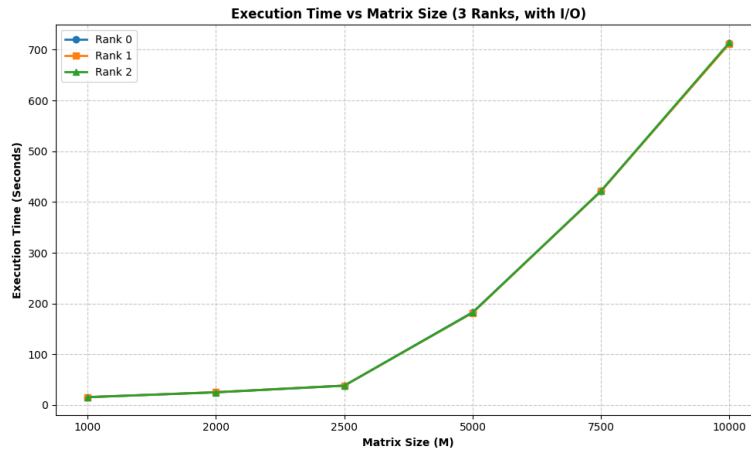


Figure 5: Execution Time vs Matrix Size.

This massive slowdown is not caused by the computational complexity of the hybrid setup, but

by an Input/Output (I/O) bottleneck. The simultaneous memory access and disk-writing operations to the shared file system by all three MPI processes, each attempting to format and save **.pgm** frames concurrently, create heavy hardware contention. This I/O saturation stalls the CPU threads, drastically inflating the overall execution time for all processes involved.

To isolate and evaluate the purely computational performance of the hybrid architecture, the disk-writing operations (I/O) were temporarily disabled. The profiling tests were then re-executed, varying the matrix size (M) while maintaining the configuration of 8 OpenMP threads per MPI task.

In the first optimization phase, all three ranks (totaling 24 threads) were deliberately confined to a single CPU socket to evaluate shared memory contention.

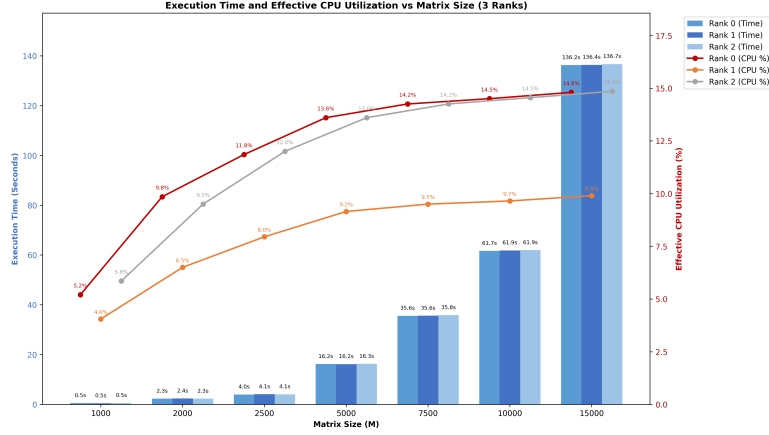


Figure 6: Execution Time and Effective CPU Utilization vs Matrix Size (3 Ranks confined to a single socket without strict pinning).

As illustrated in Figure 6, eliminating the I/O bottleneck yields a drastic reduction in execution times, dropping from over 700 seconds to approximately 136 seconds for $M = 15000$. This result is closely aligned with the 123 seconds observed in the Assignment 1 baseline, which, notably, was executed using only a single process. However, the Intel VTune profiling reveals a significant hardware-level anomaly: the Effective CPU Utilization is highly asymmetric. Rank 1 exhibits a drastically lower efficiency percentage compared to Ranks 0 and 2. Since the three processes execute the exact same computational volume, this discrepancy is not due to algorithmic imbalance.

Instead, because the 24 threads were executed within a single socket without strict hardware pinning, the Linux operating system attempted to optimize execution management by occasionally migrating computations from heavily utilized cores to colder ones. When a thread wakes up on a newly assigned core and attempts to compute the 2D stencil, it experiences a "cache miss" [6]. Since each physical core relies on its own local level of cache, the required grid data is no longer immediately available. The processor is therefore forced to fetch the data all the way from the main RAM. This memory access operation is extremely slow and causes a memory stall, effectively freezing the CPU while it waits for the data to arrive. This phenomenon predominantly affected the threads of Rank 1, and these continuous RAM access stalls are the exact reason why its Effective CPU Utilization is severely degraded compared to the other two processes.

To validate this memory contention hypothesis, a second profiling test was conducted by physically distributing the ranks across the node's Non-Uniform Memory Access (NUMA) architecture. Rank 0 and Rank 2 were strictly pinned to Socket 0, while Rank 1 was granted exclusive access to Socket 1.

The results in Figure 7 perfectly illustrate the critical impact of memory bandwidth. By isolating Rank 1 on a separate socket, it gained unhindered access to the memory controller, allowing it to finalize its local stencil computations significantly faster than the other two processes, which were still competing for cache and RAM bandwidth on Socket 0 (e.g., executing in roughly 88 seconds versus 150 seconds).

However, this topology introduces a counter-intuitive phenomenon regarding the recorded CPU efficiency, caused by the global synchronization barrier. Because the overall application relies on `MPI_Finalize` to terminate safely, Rank 1 completes its physical calculations early and subsequently

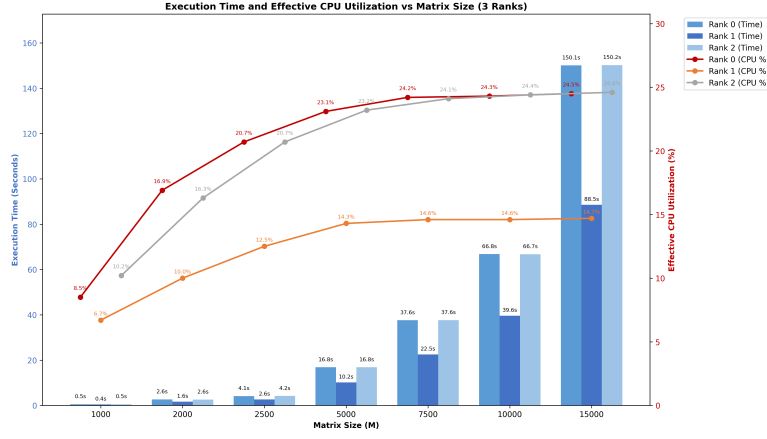


Figure 7: Execution Time and Effective CPU Utilization vs Matrix Size (Rank 1 isolated on a separate socket).

enters a prolonged idle spin-wait state, waiting for Ranks 0 and 2 to finish. Since VTune averages the Effective CPU Utilization over the entire lifespan of the process, this extended idle period drastically dilutes Rank 1’s final metric. This apparent efficiency degradation is, in reality, a direct proof of its superior execution speed: hardware isolation allowed the rank to severely outpace the others, trading measured CPU utilization for raw computational velocity.

3 Assignment 3

3.1 Introduction

This assignment extends the damped wave simulator from parts 1–2 with color. The physics, discretisation, and MPI/OpenMP structure are unchanged: three MPI ranks each run one independent scenario (a single impulse; two impulses starting together; two impulses with the second delayed to $N/7$), each parallelised with OpenMP. What changes is the output stage: each timestep’s matrix is sent to the GPU, mapped to an RGB triplet per pixel by a CUDA kernel, and saved as a P3 (ASCII) PPM frame instead of a grayscale PGM. Bright, saturated colors mark the wave’s extrema, colors fade along the rising/falling fronts, and calm (near-zero) regions are rendered white.

3.2 Architecture

The three MPI ranks [9] map naturally onto the three required scenarios: each rank owns a complete, independent grid and runs one scenario end to end, so the only collective call needed is a final `MPI_Allreduce` on success/failure. Within each rank, `compute_next` (the same 5-point Laplacian stencil as parts 1–2 – each point’s next value is computed from itself and its 4 orthogonal neighbors via central differences) is parallelised with OpenMP [8], using `schedule(static)` for the same NUMA first-touch reason argued in part 1: with a fixed thread-to-row mapping, the memory each thread first writes (and therefore where the OS physically places it) matches the memory that same thread repeatedly accesses on every subsequent call. After each timestep, the rank sends its matrix to the GPU [11]: a kernel normalizes each value against the frame’s own peak magnitude (found via a GPU-side reduction [12]), maps it to a color (white near zero, fading to bright red/orange for positive values and blue for negative, saturating at the wave’s extrema), and the result is written to disk as an ASCII PPM frame while the original scalar matrix continues to drive the simulation. Figure 8 shows one rendered frame. Because the job requests a single GPU (`--gres=gpu:1`) for all three ranks, the GPU is a shared resource – a design point this report returns to in Section 3.8.

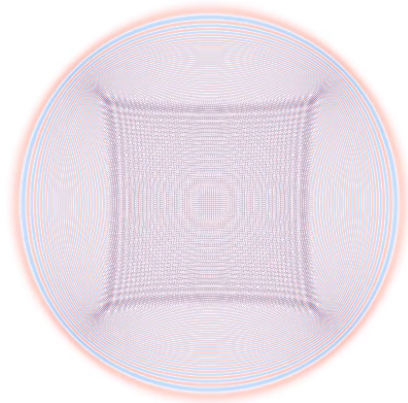


Figure 8: A rendered frame (sim1, late in the simulation): bright extrema, fading fronts, white calm regions.

3.3 Cluster hardware

All experiments ran on Politecnico di Torino’s HPC@PoliTO cluster (Legion), specifically its EDU partitions – reserved for coursework, as opposed to the open-access or research-group-restricted partitions also present on the cluster. Of the four EDU partitions, only two have GPUs: `edu_a40` and `edu_v100`, confirmed live via `sinfo -o "%P %D %G"` (the other two, `edu_skylake` and `edu_sapphire`, are CPU-only). `edu_a40` nodes have 2 physical CPU sockets, 24 physical cores per socket (48 physical cores/node), doubled to 96 logical cores/node by SMT (2 hardware threads per core) – confirmed via `sinfo`’s `socket/core/thread` fields, since the cluster’s own documentation lists only the 48 physical cores – and 4×A40 GPUs; `edu_v100` nodes have 32 cores and 4×V100 GPUs. Both are used in this report: `edu_a40` for all main results, and `edu_v100` as a cross-hardware check in Section 3.6.

3.4 Metrics and the grid-size sweep

Beyond wall-clock time, the program times two phases directly: `color_time` (GPU colorization + PPM write, per frame) and `compute_time` (the OpenMP stencil update). `color_time` is further split into five sub-phases – H2D copy, kernel, D2H copy, ASCII formatting, and disk write – introduced in Section 3.5. Figure 9 sweeps grid size M from 128 to 4096 (`edu_a40`, 4 threads, 50 steps). Execution time approaches ideal $O(M^2)$ scaling as M grows: $128 \rightarrow 256$ costs only $2.1\times$ (fixed per-run overhead still dominant at small M), while $2048 \rightarrow 4096$ costs $4.1\times$ – essentially the ideal quadratic ratio. The right panel shows GPU colorize+write dominating total time throughout: at $M = 4096$ it is $\sim 97.6\%$ of total time, versus `compute_next` at $\sim 2.4\%$. Both phases are $O(M^2)$, so this ratio stays roughly constant across the whole sweep ($\sim 97.6\text{--}97.7\%$ from $M = 256$ upward; only the smallest grid, $M = 128$, deviates, where fixed per-run overhead is large enough to matter).

We capped the sweep at $M = 4096$ rather than matching part 1’s $M = 15000$: at $M = 4096$ an ASCII PPM frame is already $\sim 100\text{--}130\text{MB}$, so scaling to $M = 15000 - (15000/4096)^2 \approx 13\times$ the pixels – would push single frames into the multi-gigabyte range across 50 steps and 3 ranks, for a sweep whose purpose (showing execution time approach ideal $O(M^2)$ scaling and locating the point where GPU colorize+write comes to dominate) was already unambiguous well before $M = 4096$.

3.5 Phase breakdown: is the ASCII formatting worth parallelizing?

Figure 10 breaks `color_time` into its five sub-phases. At $M = 4096$ (averaged over 3 ranks): `write` (the `fwrite` call) is $\sim 94.6\%$ of `color_time`, `format` (the ASCII byte-to-digit conversion) is $\sim 4.3\%$, and H2D copy + kernel + D2H copy combined are $\sim 1\%$. The same lopsided pattern holds at every grid size swept, not only at $M = 4096$.

This directly answers a question we considered early on: is it worth parallelizing the ASCII-formatting step with OpenMP? It is not – formatting is only $\sim 4\%$ of `color_time`, so even a theoretical zero-overhead parallelization would save a few percent of total time at best, not worth the added

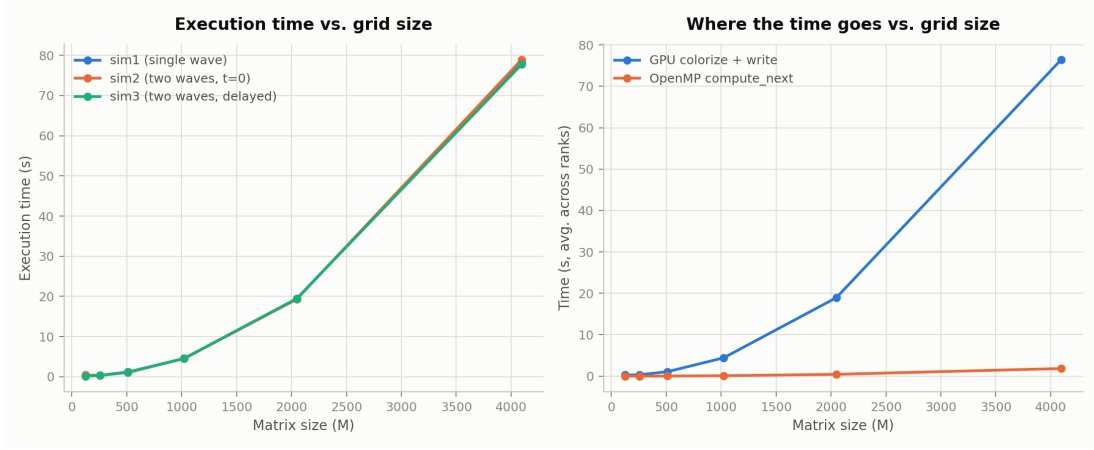


Figure 9: Execution time and its GPU/OpenMP split, varying grid size M (4 threads, 3 ranks sharing 1 GPU).

complexity (the output is variable-width per pixel, so parallel writers would need either a prefix-sum pass to compute offsets first, or a switch to fixed-width padding). We measured before optimizing, and the measurement said no.

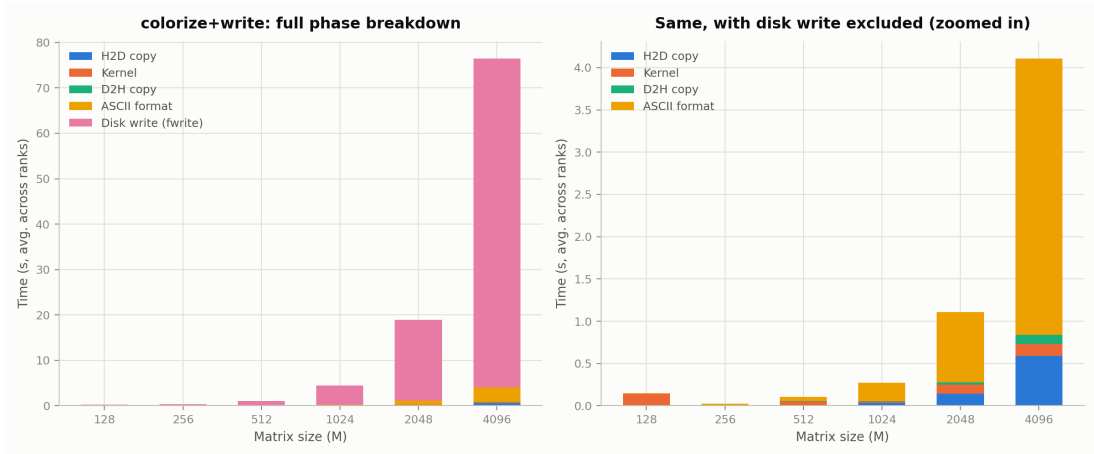


Figure 10: colorize+write split into its 5 phases vs. grid size. Left: full stack (write’s dominance is the point, not log-scaled). Right: same data with write excluded, zoomed in.

3.6 Where does the write time actually go? Nsight Systems, cross-checked on two GPUs

Nsight Compute (kernel-level occupancy/throughput metrics) was attempted but is unavailable on this cluster: it returns `ERR_NVGPUCTRPERM` – GPU performance-counter access is admin-only, a driver-level restriction no user-side flag can work around. Nsight Systems was used instead, and it revealed something our own coarse `write_s` timer could not: that timer wraps `fopen+fwrite+fclose` as one block, and Nsight Systems’ OS Runtime Summary lets us split it apart. On `edu_a40`: `fwrite` itself is cheap ($\sim 0.6\%$ of total OS-runtime time), while `fclose` is $\sim 18.3\%$ – roughly $30\times$ more expensive than the write call it follows. The two largest categories by far, `epoll_wait` ($\sim 52\%$) and `poll` ($\sim 25\%$), we initially guessed reflected ranks idling on MPI’s progress engine. A follow-up run with MPI event tracing enabled rules that out directly: summed across `MPI_Init`, `MPI_Finalize`, and the one real collective (`MPI_Allreduce`), application-level MPI time is $\sim 0.56s$ per rank – under 2% of the combined `epoll_wait+poll` time ($\sim 28.4s$). `MPI_Allreduce` itself is $13.7ms$. The better-supported explanation, from a companion VTune Threading run on the same job: its top wait objects include InfiniBand verbs streams (`/dev/infiniband/verbs0`), suggesting OpenMPI’s UCX transport stack is initialized and

idly polling in the background for the whole process lifetime regardless of how little it is actually used – not the application’s own collective calls, but the runtime’s transport layer underneath them. We report this as the better-supported hypothesis, not a fully isolated cause. To check whether the OS-runtime pattern itself is an artifact of one specific GPU, we repeated the identical run on `edu_v100` – an older GPU generation with a slower host interconnect (the whole production run was $\sim 3\times$ slower end-to-end on V100, consistent with H2D transfer time alone rising from $\sim 52\%$ to $\sim 67\%$ of GPU time). Despite the large hardware difference, the OS-runtime split is nearly identical (Figure 11): `fclose` $\sim 17.9\%$, `fwrite` $\sim 0.4\%$, `epoll_wait`+`poll` $\sim 76\%$ combined on both. This is good evidence the bottleneck is I/O/synchronization behavior, not GPU-specific.

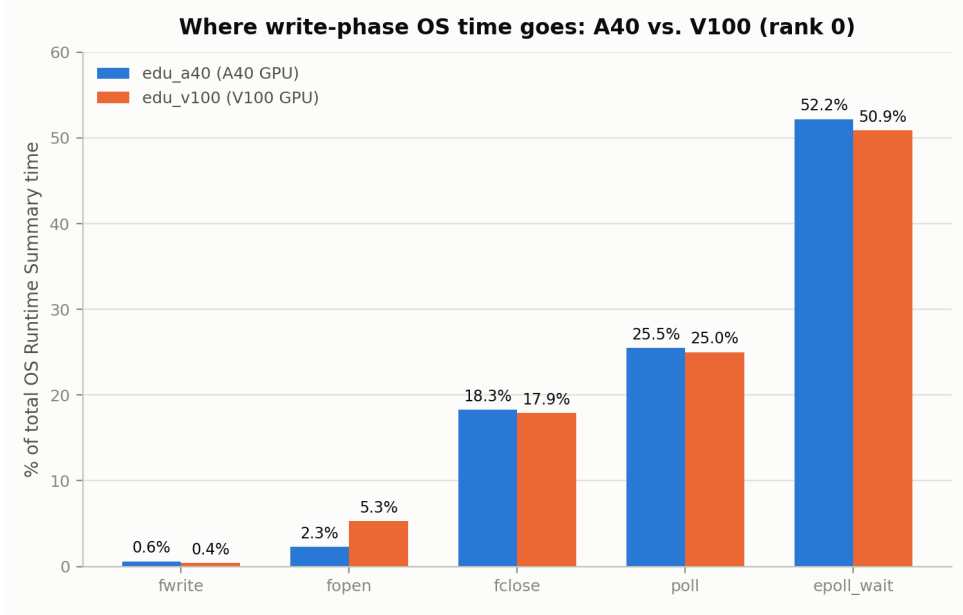


Figure 11: OS Runtime Summary breakdown (rank 0), `edu_a40` vs. `edu_v100` – nearly identical despite very different GPU hardware.

3.7 OpenMP thread scaling and CUDA block size

Figure 12 sweeps OpenMP thread count 1–32 at $M = 4096$ (the full logical-core range of one `edu_a40` rank, 96/3). Both the full-pipeline and `compute_next`-only curves are essentially flat, never exceeding $\sim 1.2\times$ speedup, with compute-only plateauing by 2 threads and staying flat through 32. This is consistent with the 5-point stencil being memory-bandwidth-bound rather than compute-bound (4 loads, 1 store, few FLOPs per point) – a small number of threads already saturates available memory bandwidth. We also considered an alternative explanation – that `compute_next` forking a fresh OpenMP thread team on every one of 300 calls/rank could plateau this curve through fork/join overhead alone, independent of memory bandwidth – and ruled it out with VTune Hotspots at $M = 4096$: `gomp_team_barrier_wait_end` does not appear among the top hotspots at either 4 or 32 threads (unlike part 1’s profile, where it was 79.4% of CPU time), and `compute_next`’s own CPU time is flat across that range (3.43s at 4 threads vs. 3.21s at 32) rather than growing with team size. This contrasts with part 1’s own OpenMP results, which showed real scaling at large thread counts, mainly because part 1 tested M up to 15000 – large enough for the stencil’s absolute cost to be interesting on its own, with no competing GPU-bound stage.

Figure 13 sweeps the CUDA kernel’s threads-per-block at $M = 4096$: time decreases roughly monotonically as block size grows, worst at 32 threads/block (~ 83 s average) and best at 1024 (~ 73 s, $\sim 12\%$ faster), with only a minor plateau between 128 and 512. Since Nsight Compute (the tool that would report actual occupancy) is unavailable on this cluster, we do not have a profiler-level explanation for this trend, only the timing result itself – the most we can say is that fewer, larger blocks amortize per-block launch and scheduling overhead across more threads. A smaller, noisier

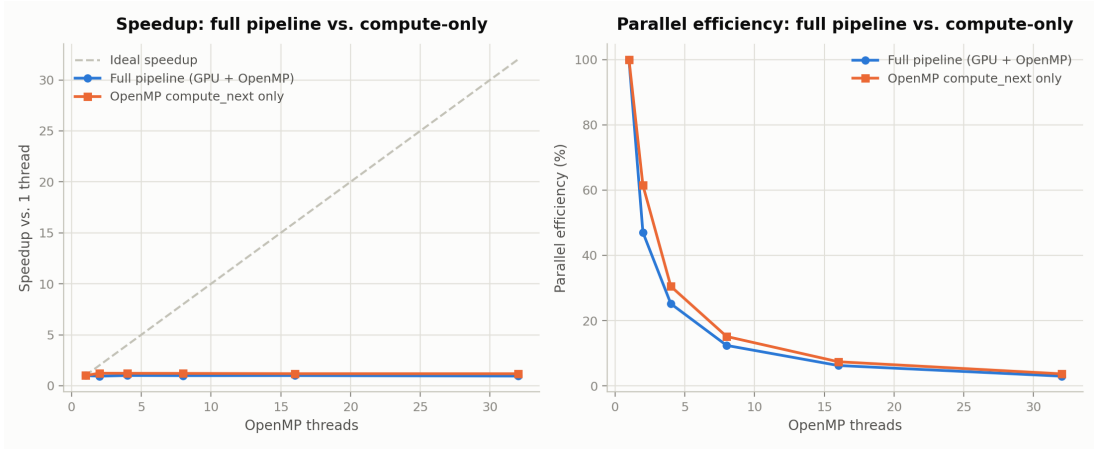


Figure 12: Speedup and efficiency vs. OpenMP thread count at $M = 4096$, full pipeline vs. compute-only.

earlier sweep at $M = 1024$ (not shown) trended the opposite way at the high end, so we do not treat this shape as independently confirmed and report only the $M = 4096$ result.

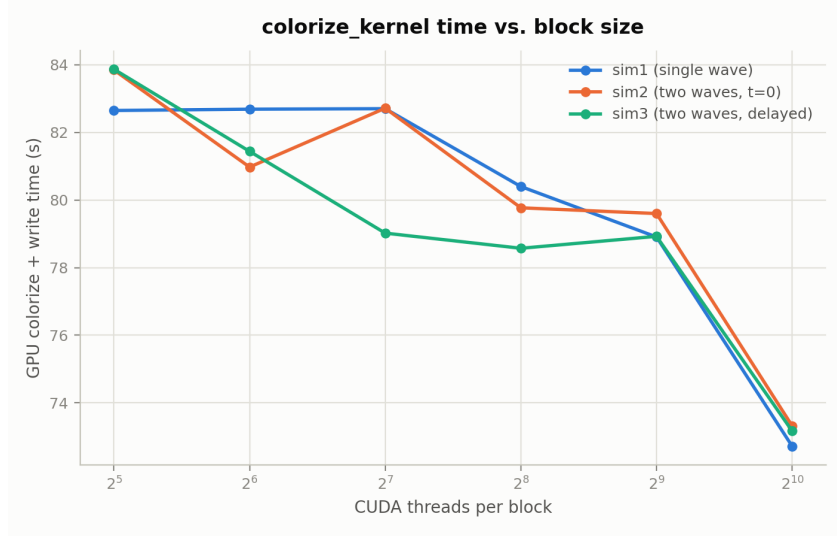


Figure 13: GPU colorize+write time vs. CUDA threads-per-block, $M = 4096$.

3.8 GPU contention across execution modes

Section 3.5 showed disk writes dominate; this raises the question of how much the design choice of sharing one GPU across 3 ranks actually costs. We compared four execution modes at identical parameters ($M = 2048$, 50 steps, 4 threads), isolating sim1's workload in each: **solo** (1 rank, exclusive GPU), **shared** (the normal 3-rank/1-GPU run), **distributed** (3 ranks, 3 dedicated GPUs on one node, `--gres=gpu:3`), and **multinode** (3 ranks on 3 separate nodes, one dedicated GPU each, communicating over the cluster's Infiniband fabric [10]). Results (Figure 14): **solo** 9.47s, **shared** 20.84s, **distributed** 21.25s, **multinode** 18.76s.

Two findings. First, dedicating a GPU per rank alone (**distributed** vs. **shared**) makes no reliable difference at all – **distributed** is actually $\sim 2\%$ slower here, well within run-to-run noise rather than a real effect either way – GPU compute-engine time-slicing is not the bottleneck. Second, **solo** is dramatically faster than every multi-rank mode (roughly half the time, $\sim 50\text{--}55\%$ less depending on which mode it's compared against), which Section 3.5's finding explains directly: **shared** and **distributed** both have 3 ranks writing frames *concurrently* to the same node; **solo** is the only mode with exactly one writer at a time, anywhere. **multinode** sits in between ($\sim 10\%$ faster than

distributed) by removing host-local contention (memory bus, PCIe root complex) but not the deeper one: all three nodes still write to the same shared network filesystem (BeeGFS/NFS, cluster-wide by design), so multinode is closer to **shared** than to **solo**. The dominant contention in this workload is concurrent I/O, not GPU compute-engine sharing – consistent with Section 3.6’s **fclose**-dominated OS Runtime profile.

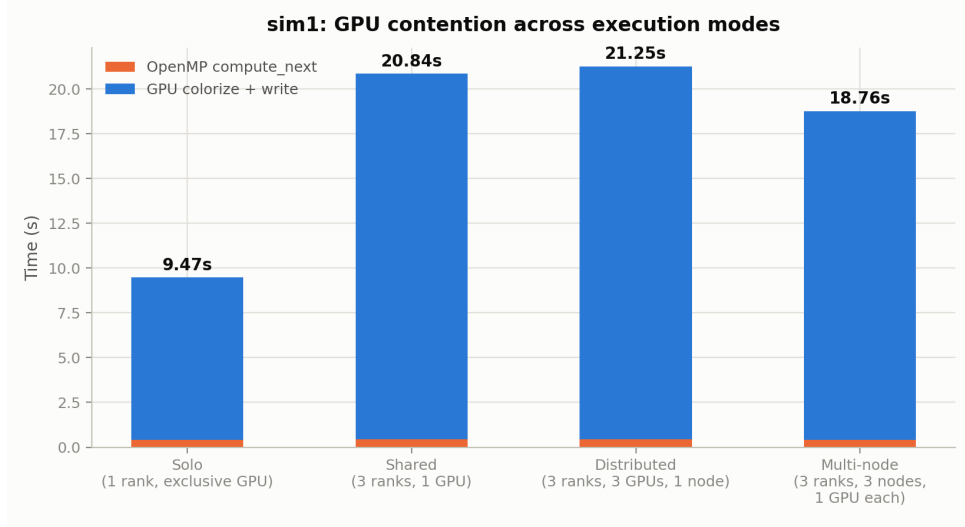


Figure 14: sim1’s execution time across four GPU/node sharing configurations, identical workload.

3.9 Conclusions

The dominant cost in this pipeline is not GPU compute (colorization kernel: $\sim 1\%$ of frame time) nor CPU compute (**compute_next**: $\sim 2\%$), but per-frame file I/O, specifically whatever happens at **fclose** time rather than the write itself – confirmed consistent across two GPU generations and reflected directly in the GPU-sharing contention results: sharing the GPU across ranks costs almost nothing, while sharing the filesystem costs a great deal. This is the throughline of the whole investigation – every optimization decision was made only after measuring, not assumed from the physics alone. Parallelizing the ASCII-formatting step was considered and rejected once the phase breakdown showed it was $\sim 4\%$ of **color_time**, not worth the added complexity. Per-frame adaptive color scaling was adopted only after raw pixel data confirmed a fixed scale reads as pale almost immediately, since a 2D wave’s peak amplitude falls off geometrically with distance from the source on top of γ -damping. And the GPU-sharing question itself was settled by directly comparing four execution modes rather than assumed from GPU specifications – the result (I/O-bound, not compute-bound) held consistently across every measurement in this report.

References

- [1] Politecnico di Torino. Hpc@polito infrastructure guide. https://hpc.polito.it/docs/hpcpolito_guide.pdf.
- [2] Intel Corporation. Hotspots analysis for CPU usage issues – intel vtune profiler user guide. <https://www.intel.com/content/www/us/en/docs/vtune-profiler/user-guide/2026-1/basic-hotspots-analysis.html>.
- [3] Intel Corporation. Threading analysis – intel vtune profiler user guide. <https://www.intel.com/content/www/us/en/docs/vtune-profiler/user-guide/2026-1/threading-analysis.html>.
- [4] Deborah T. Marr, Frank Binns, David L. Hill, Glenn Hinton, David A. Koufaty, J. Alan Miller, and Michael Upton. Hyper-threading technology architecture and microarchitecture. *Intel Technology Journal*, 2002. <https://www.intel.com/content/dam/www/public/us/en/documents/research/2002-vol06-iss-1-intel-technology-journal.pdf>.
- [5] OpenMP Architecture Review Board. OpenMP application programming interface, version 5.0. Technical report, OpenMP Architecture Review Board, November 2018. <https://www.openmp.org/spec-html/5.0/openmp.html>.
- [6] Edgar Ernesto Sanchez Sanchez. 07_cache_mem.pdf course slides, 2026.
- [7] Edgar Ernesto Sanchez Sanchez. 10_parallel_architectures.pdf course slides. 2026.
- [8] Edgar Ernesto Sanchez Sanchez. 14_openmp.pdf course slides. 2026.
- [9] Edgar Ernesto Sanchez Sanchez. 16_mpi_intro.pdf course slides. 2026.
- [10] Edgar Ernesto Sanchez Sanchez. 17_mpi_virtual_topologies.pdf course slides. 2026.
- [11] Edgar Ernesto Sanchez Sanchez. 18_cuda.pdf course slides. 2026.
- [12] Edgar Ernesto Sanchez Sanchez. 19_gpu_programming_intro.pdf course slides. 2026.
- [13] Wikipedia. Finite difference method. https://en.wikipedia.org/wiki/Finite_difference.